

Pyspark to Sql

```
from pyspark.sql import SparkSession
```

```
from pyspark.sql import functions as F
```

```
from pyspark.sql.types import (
```

```
    StructType,
```

```
    StructField,
```

```
    StringType,
```

```
    IntegerType,
```

```
    DoubleType,
```

```
    TimestampType
```

```
)
```

```
def get_spark_session(app_name: str = "OrderProcessingETL") -> SparkSession:
```

```
    """Initialize and configure SparkSession."""
```

```
    return (
```

```
        SparkSession.builder
```

```
        .appName(app_name)
```

```
        .master("local[*]") # Uses all local cores; on cluster, omit or set via spark-submit
```

```
        .config("spark.sql.adaptive.enabled", "true") # Adaptive Query Execution (AQE)
```

```
        .config("spark.sql.parquet.compression.codec", "snappy")
```

```
        .getOrCreate()
```

```
    )
```

```
# -----
```

```
# 1. EXTRACT
```

```
# -----
```

```
def define_raw_schema() -> StructType:
    """Enforce explicit schema on read to avoid expensive schema inference."""
    return StructType([
        StructField("order_id", StringType(), nullable=True),
        StructField("customer_id", StringType(), nullable=True),
        StructField("order_timestamp", StringType(), nullable=True),
        StructField("item_name", StringType(), nullable=True),
        StructField("quantity", IntegerType(), nullable=True),
        StructField("unit_price", DoubleType(), nullable=True),
        StructField("discount_pct", DoubleType(), nullable=True),
        StructField("status", StringType(), nullable=True)
    ])

```

```
def extract_orders(spark: SparkSession, file_path: str, schema: StructType):
    """
    Extracts raw data with schema enforcement.
    Can read from S3 (s3a://), HDFS (hdfs://), ADLS (abfss://), or local disk.
    """
    return (
        spark.read
        .format("json")
        .schema(schema)
        .load(file_path)
    )

```

```
# -----

```

2. TRANSFORM

def transform_orders(df):

"""

Cleanses, normalizes, deduplicates, and enriches order records.

"""

Step A: Drop records missing critical primary identifiers

cleaned_df = df.filter(

F.col("order_id").isNotNull() & F.col("customer_id").isNotNull()

)

Step B: Remove duplicate records based on natural primary key

deduped_df = cleaned_df.dropDuplicates(["order_id"])

Step C: Normalize strings, parse timestamps, and handle nulls

standardized_df = (

deduped_df

Parse timestamp string into Spark TimestampType

.withColumn("order_time", F.to_timestamp(F.col("order_timestamp"), "yyyy-MM-dd HH:mm:ss"))

Standardize strings

.withColumn("item_name", F.trim(F.lower(F.col("item_name"))))

.withColumn("status", F.trim(F.upper(F.col("status"))))

Impute missing quantity or discount with safe defaults

.withColumn("quantity", F.coalesce(F.col("quantity"), F.lit(1)))

.withColumn("discount_pct", F.coalesce(F.col("discount_pct"), F.lit(0.0)))

.withColumn("unit_price", F.coalesce(F.col("unit_price"), F.lit(0.0)))

)

```

# Step D: Derive analytical and business metrics

enriched_df = (
    standardized_df
    # Gross revenue = quantity * unit_price
    .withColumn("gross_amount", F.round(F.col("quantity") * F.col("unit_price"), 2))
    # Net revenue after discount
    .withColumn(
        "net_amount",
        F.round(
            F.col("gross_amount") * (1.0 - (F.col("discount_pct") / 100.0)),
            2
        )
    )

    # Derive date partitions
    .withColumn("order_date", F.to_date(F.col("order_time")))
    .withColumn("order_year", F.year(F.col("order_time")))
    .withColumn("order_month", F.month(F.col("order_time")))

    # Flag large orders
    .withColumn("is_high_value", F.when(F.col("net_amount") >= 500.0,
True).otherwise(False))
)

# Step E: Prune columns not needed downstream

final_df = enriched_df.select(
    "order_id",
    "customer_id",
    "order_time",

```

```
    "item_name",
    "quantity",
    "unit_price",
    "gross_amount",
    "net_amount",
    "is_high_value",
    "status",
    "order_year",
    "order_month"
)
```

```
return final_df
```

```
# -----
```

```
# 3. LOAD
```

```
# -----
```

```
def load_to_parquet(df, target_path: str, partition_cols: list = None):
```

```
    """
```

```
    Persists DataFrame to Parquet format using Snappy compression and partitioning.
```

```
    """
```

```
    writer = (
```

```
        df.write
```

```
        .mode("overwrite") # Options: overwrite, append, errorIfExists, ignore
```

```
        .format("parquet")
```

```
        .option("compression", "snappy")
```

```
    )
```

```

__if partition_cols:
__    writer = writer.partitionBy(*partition_cols)

__writer.save(target_path)

# -----
# PIPELINE EXECUTION HARNESS
# -----

def run_pipeline():

__    spark = get_spark_session()

__    try:

__        # Create mock raw input to demonstrate execution immediately

__        mock_raw_data = [

__            {"order_id": "ORD001", "customer_id": "CUST_A", "order_timestamp": "2026-03-01
10:15:30", "item_name": " MECHANICAL KEYBOARD ", "quantity": 2, "unit_price": 75.50,
"discount_pct": 10.0, "status": "completed"},

__            {"order_id": "ORD002", "customer_id": "CUST_B", "order_timestamp": "2026-03-01
11:20:00", "item_name": "Mousepad XL", "quantity": 1, "unit_price": 25.00,
"discount_pct": None, "status": "COMPLETED"},

__            {"order_id": "ORD002", "customer_id": "CUST_B", "order_timestamp": "2026-03-01
11:20:00", "item_name": "Mousepad XL", "quantity": 1, "unit_price": 25.00,
"discount_pct": None, "status": "COMPLETED"}, # Duplicate

__            {"order_id": None, "customer_id": "CUST_C", "order_timestamp": "2026-03-02
09:00:00", "item_name": "USB Cable", "quantity": 1, "unit_price": 10.00, "discount_pct":
0.0, "status": "completed"}, # Corrupt primary key

__            {"order_id": "ORD003", "customer_id": "CUST_D", "order_timestamp": "2026-04-15
14:45:10", "item_name": "4K Monitor", "quantity": 2, "unit_price": 320.00,
"discount_pct": 5.0, "status": "pending"},

__        ]

```

```
__ raw_input_dir = "/tmp/spark_etl_demo/raw_orders"
__ target_parquet_dir = "/tmp/spark_etl_demo/gold_orders"

__ # 1. Setup mock source
__ raw_schema = define_raw_schema()
__ spark.createDataFrame(mock_raw_data,
schema=raw_schema).write.mode("overwrite").json(raw_input_dir)

__ # 2. Extract
__ raw_df = extract_orders(spark, raw_input_dir, raw_schema)

__ # 3. Transform
__ clean_df = transform_orders(raw_df)

__ print("\n--- Transformed Preview ---")
__ clean_df.show(truncate=False)

__ # 4. Load
__ load_to_parquet(
__     df=clean_df,
__     target_path=target_parquet_dir,
__     partition_cols=["order_year", "order_month"]
__ )

__ # 5. Verification: Read back from Parquet to ensure integrity
__ written_df = spark.read.parquet(target_parquet_dir)
__ print(f"\nSuccessfully wrote {written_df.count()} records to {target_parquet_dir}")
```

```
__finally:  
    __spark.stop()
```

```
if __name__ == "__main__":  
    __run_pipeline()
```